Text Generation e Chatbot

Lorenzo Chesta
A.A 2025/2026

Indice

1. Introduzione 3
2. Principali approcci alla Text Generation nella storia 4
3. Large Language Model e Chatbot 11
4. Allucinazioni e strategie di mitigazione 18
5. Conclusioni 35

1. Introduzione

Negli ultimi anni, il processamento del linguaggio naturale ha visto uno sviluppo significativo. In particolare, nell'ambito della Text Generation, l'emergere dei chatbot basati sui Large Language Model (LLM) ha segnato un punto di svolta: si è passati dai primi sistemi basati su regole, caratterizzati da un'estrema rigidità, a modelli moderni capaci di cogliere le sfumature del linguaggio e sostenere conversazioni coerenti.

Tuttavia, è importante capire fino a che punto ci si può fidare di un chatbot, o in generale di un LLM.

Lo scopo di questo approfondimento, perciò, è duplice:

- la prima parte è dedicata a ripercorrere sinteticamente l'evoluzione degli approcci alla Text Generation, soffermandosi in particolare sulle innovazioni che ne hanno permesso il salto di qualità;
- la seconda parte, più ampia, si concentra sui chatbot LLM-based. In particolare, vengono esaminati il fenomeno delle allucinazioni e le diverse tecniche per mitigarlo.

2. Principali approcci alla Text Generation nella storia

2.1 Approccio rule-based

I primi sistemi di generazione testuale si basavano sull'idea di mappare il linguaggio naturale su un insieme finito di strutture sintattiche. Dal punto di vista implementativo, ciò era possibile grazie a tecniche di pattern matching, gruppi di cattura e regole di riformulazione.

Uno dei primi programmi basati su regole che simulava la comunicazione in linguaggio naturale tra esseri umani e macchine è ELIZA [1]. Di seguito si riporta un breve estratto di un'interazione tra un essere umano ed ELIZA, che simulava uno psicoterapeuta:

```
User:  I am unhappy.
ELIZA: DO YOU THINK COMING HERE WILL HELP YOU NOT TO BE UNHAPPY
User:  I need some help, that much seems certain.
ELIZA: WHAT WOULD IT MEAN TO YOU IF YOU GOT SOME HELP
User:  Perhaps I could learn to get along with my mother.
ELIZA: TELL ME MORE ABOUT YOUR FAMILY
User:  My mother takes care of me.
ELIZA: WHO ELSE IN YOU FAMILY TAKES CARE OF YOU
User:  My father.
ELIZA: YOUR FATHER
User:  You are like my father in some ways.
```

Fig. 1: dialogo tra un utente ed ELIZA.

ELIZA operava analizzando le frasi digitate dall'utente sulla base di un insieme di regole gerarchiche contenute in uno script. Nello specifico:

1. ELIZA scansionava il testo dell'utente alla ricerca di parole chiave (keywords). Se la frase conteneva più parole chiave, il sistema selezionava quella con il rango di priorità più elevato, deciso manualmente dal progammatore;
2. una volta scelta una parola chiave, ELIZA applicava una regola di decomposizione, cioè un template che suddivideva la frase dell'utente in segmenti specifici, isolando le parti del testo che circondavano la parola chiave stessa;
3. a ogni regola di decomposizione erano associate una o più regole di riassemblaggio, che indicavano al sistema come ricomporre i segmenti isolati per generare una risposta. Durante questo processo, venivano effettuate sostituzioni automatiche dei pronomi (ad esempio trasformando la prima persona nella seconda) per mantenere la coerenza nel dialogo.

Il punto di forza dei sistemi rule-based era la facile interpretabilità: dato che le regole venivano definite dall'uomo, ogni "decisione" del sistema poteva essere facilmente ricondotta ad una regola specifica.

Tuttavia, sistemi di questo tipo non riuscivano a manipolare testo che non corrispondesse perfettamente alle regole predefinite. Inoltre erano difficili da scalare: per far sì che un sistema rule-based fosse in grado di riconoscere nuovi pattern era necessario aggiungere nuove regole, con il rischio di cadere in contraddizione con quelle definite in precedenza.

2.2 Approccio statistico

I limiti dell'approccio rule-based per la Natural Language Generation sono stati superati dall'approccio statistico, che trova la principale applicazione nei modelli a *n*-grammi, dove per *n*-gramma si intende una sequenza di *n* parole. L'approccio statistico non prevede la definizione manuale di regole, in quanto il modello linguistico "apprende" direttamente analizzando i corpora. Questo si traduce in una maggiore scalabilità del modello.

L'idea alla base dell'approccio statistico è stimare quale parola ha la maggiore probabilità di comparire dopo una determinata sequenza. In pratica il modello calcola una probabilità condizionata [2]:

$$P(w_n|w_{1:n-1}),$$

dove il termine $w_{1:n-1}$ indica tutte le parole $w_1, w_2, \dots, w_{n-1}$ che precedono w_n (da predire).

Per rendere il calcolo di $w_{1:n-1}$ computazionalmente sostenibile si sfrutta l'assunzione di Markov, secondo cui la probabilità di una parola dipende solo dalle $N-1$ parole precedenti, e si ottiene la seguente approssimazione:

$$P(w_n|w_{1:n-1}) \approx P(w_n|w_{n-N+1:n-1}).$$

In altre parole, in fase di generazione il modello è auto-regressivo: non produce l'intero testo simultaneamente, ma seleziona una parola alla volta campionando dalla distribuzione di probabilità calcolata. Una volta generata la parola w_n, essa viene aggiunta al contesto per predire w_{n+1}.

Dall'assunzione di Markov si intuisce il limite principale dell'approccio statistico applicato alla NLG: la mancanza di memoria a lungo raggio. Dato che un *n*-gramma considera solo una finestra limitata di parole precedenti, il testo generato tende a mostrare una coerenza locale (le parole vicine sono corrette dal punto di vista sintattico) ma non globale (il discorso non ha un senso logico complessivo).

2.3 Approccio neurale

2.3.1 RNN

Il limite della “finestra fissa” dell’approccio statistico è stato superato dalle Recurrent Neural Networks, che trattano sequenze temporali ricorrenti, come le parole.

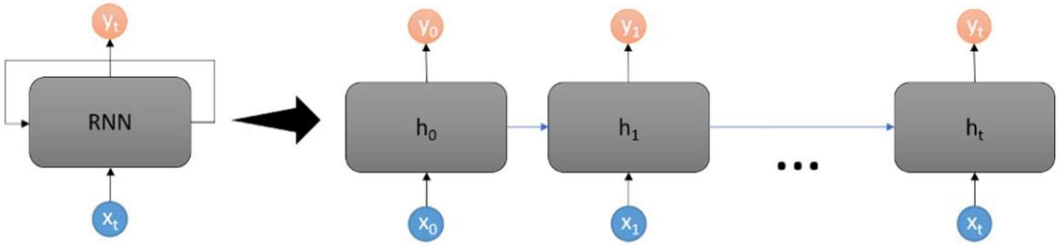

Fig. 2: architettura di una RNN.

Il punto di forza di una RNN è lo stato nascosto (hidden state), che funge da memoria interna. Ad ogni passo temporale t (ovvero per ogni parola della sequenza nel caso della text generation), la rete non riceve solo l’input attuale x_t, ma anche lo stato nascosto del passo $t-1$. In pratica lo stato nascosto porta con sé l’informazione delle parole precedenti (contesto).

Le RNN sono però affette dal problema del vanishing gradient: propagando a ritroso l’errore attraverso i vari step temporali durante l’addestramento (Backpropagation Through Time) si verifica una moltiplicazione a catena di derivate parziali. Se tali derivate sono minori di 1, il gradiente nei primi layer della rete tende a 0. Questo si traduce in una perdita di relazioni a lungo raggio della RNN.

Le RNN presentano anche un problema dal punto di vista computazionale: data la loro natura sequenziale (ovvero che lo stato h_t dipende da h_{t-1}), non è possibile parallelizzare l’addestramento, rendendo quest’ultimo estremamente lento per lunghe sequenze o per dataset di grandi dimensioni.

2.3.2 LSTM

Il problema del vanishing gradient tipico delle RNN è stato mitigato dalle LSTM (Long Short-Term Memory): esse hanno introdotto il Cell State C_t, una struttura di memoria che affianca lo stato nascosto. In questo modo, lo stato nascosto funge da memoria a breve termine, mentre il Cell State da memoria a lungo termine.

L’idea di fondo è che il Cell State svolga il ruolo di “corsia preferenziale” per l’informazione: esso attraversa l’intera catena temporale subendo solo lievi modifiche lineari (somme e sottrazioni), evitando le ripetute moltiplicazioni che causano la “scomparsa” del gradiente. Queste modifiche sono regolate da gates, funzioni

matematiche che ad ogni step decidono quanta informazione deve essere mantenuta, aggiunta o rimossa.

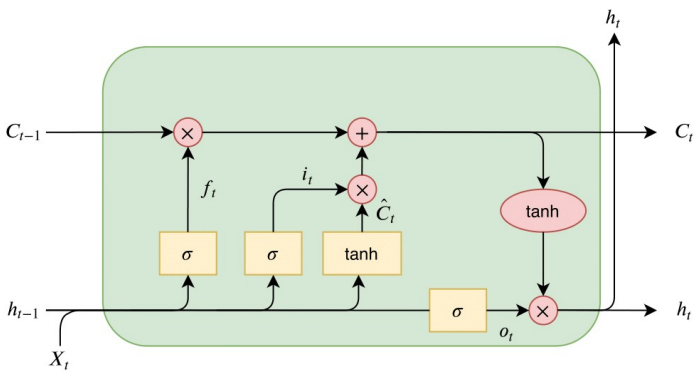

Fig. 3: architettura di una LSTM.

Tramite questo meccanismo le LSTM riescono a preservare le dipendenze anche tra parole molto distanti in una sequenza.

Tale capacità delle LSTM è stata la base dell'architettura Encoder-Decoder (o Sequence-to-Sequence), che prevede di suddividere il compito della Text Generation tra due componenti distinte:

- l'Encoder: elabora la sequenza di input parola dopo parola, aggiornando progressivamente il proprio stato interno. Al termine della lettura, l'ultimo stato (vettore di contesto) dovrebbe idealmente contenere una rappresentazione compressa dell'intero significato della frase;
- il Decoder: riceve il vettore di contesto dall'Encoder e lo utilizza come stato iniziale per iniziare a generare la sequenza di output in modo auto-regressivo.

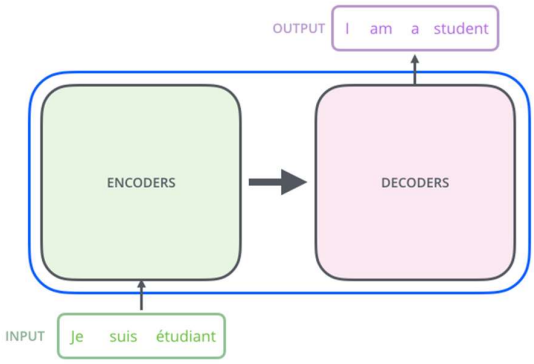

Fig. 4: architettura Encoder-Decoder per la Machine Translation.

Nonostante questo approccio abbia rappresentato un passo in avanti nell'ambito della Text Generation, esso presenta un limite strutturale: la compressione dell'informazione di una frase lunga in un unico vettore di dimensione fissa causava spesso una degradazione della qualità del testo generato (problema del "collo di bottiglia").

2.4 Transformer

Nel 2017 viene presentata una nuova architettura: il Transformer [3]. Esso ha rappresentato una vera e propria rivoluzione nel campo del NLP, segnando l'abbandono delle reti ricorrenti. Infatti, mentre una RNN elabora un token (parola) alla volta, un Transformer permette un'elaborazione in parallelo, sfruttando la GPU (Graphics Processing Unit).

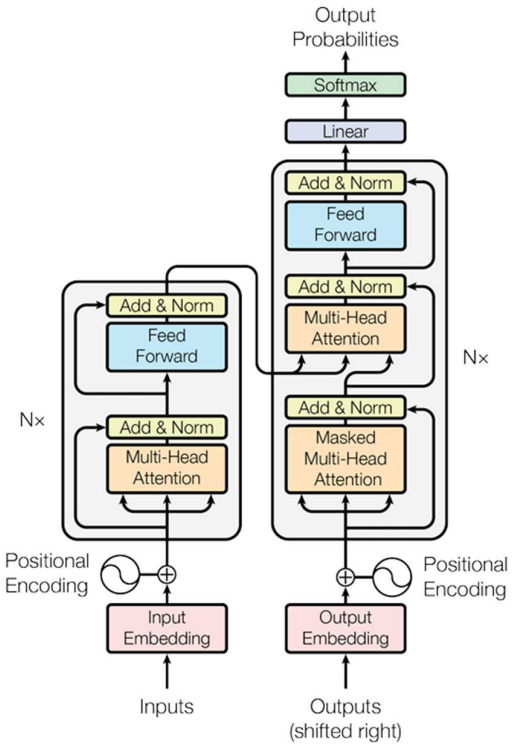

Fig. 5: architettura Transformer.

Il funzionamento di un Transformer è interamente basato sul meccanismo dell'Attenzione.

2.4.1 Self-Attention

La Self-Attention permette ad una parola di una sequenza di interagire con tutte le altre, indipendentemente dallo loro distanza all'interno della sequenza.

Si consideri la seguente frase:

"The animal didn't cross the street because it was too tired".

Per un essere umano è immediato constatare che *"it"* si riferisce a *"animal"* e non a *"street"*. Per un modello linguistico invece no, soprattutto vista la distanza tra *"it"* e *"animal"* all'interno della frase. Il meccanismo dell'Attenzione permette alla parola *"it"* di dare maggiore peso alla parola *"animal"*, identificando correttamente il riferimento logico.

Per calcolare l'Attenzione, ogni token viene mappato in 3 spazi vettoriali distinti: Query (Q), Key (K) e Value (V). L'Attenzione si ottiene come somma pesata dei valori (values), dove il peso associato ad ogni valore è determinato da una funzione di compatibilità che misura l'affinità tra la query e la relativa key. Matematicamente, questa affinità è espressa dal prodotto scalare tra i due vettori, il quale viene scalato per evitare che valori troppo elevati portino a problemi di instabilità numerica durante l'addestramento.

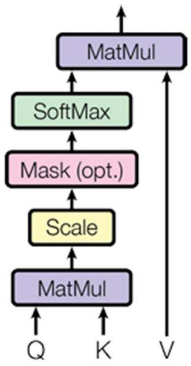

Fig. 6a: Scaled Dot-Product Attention.

2.4.2 Multi-Head Attention

Per poter catturare le diverse sfumature di una sequenza (dipendenze grammaticali, riferimenti dei pronomi, ecc), i Transformer utilizzano la Multi-Head Attention (MHA). Dal punto di vista implementativo, la MHA proietta l'input in diversi sottospazi di rappresentazione, eseguendo in parallelo più "teste" (Heads). Ogni Head si concentra su un aspetto diverso della frase. I risultati di tutte le Heads vengono poi concatenati e proiettati linearmente per ottenere un unico vettore di output della dimensione originale.

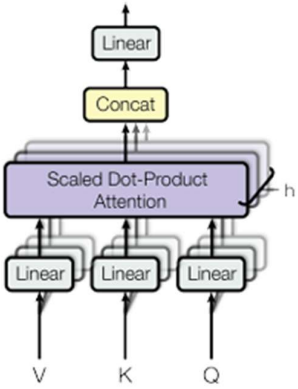

Fig. 6b: Multi-Head Attention.

2.4.3 Encoder e Decoder

La MHA viene utilizzata dal Transformer anche per far comunicare due blocchi fondamentali: l'Encoder e il Decoder.

L'Encoder riceve una sequenza di input $(x_1, \dots, x_n)$ e la mappa in una rappresentazione continua $z = (z_1, \dots, z_n)$.

Il Decoder genera una sequenza di output $(y_1, \dots, y_m)$ a partire da z, generando un elemento alla volta.

L'architettura Encoder-Decoder è stata introdotta per il task di Machine Translation, dove l'Encoder elabora la lingua di partenza e il Decoder genera la traduzione.

Tuttavia, i moderni Large Language Models (LLM) hanno semplificato tale architettura, poiché usano solo il Decoder (sono "Decoder-only").

3. Large Language Model e Chatbot

3.1 LLM

I Large Language Models (LLM) sono reti neurali profonde basate sull'architettura Transformer addestrate su enormi quantità di dati testuali per capire e generare il linguaggio naturale.

Il primo passo verso l'implementazione di un LLM è rappresentato dal paradigma GPT (Generative Pre-Training) descritto da OpenAI nel paper "*Improving Language Understanding by Generative Pre-Training*" [4].

Secondo quanto riportato nel paper, OpenAI ha introdotto GPT per rispondere a diverse sfide nell'ambito del NLP, tra cui:

- la scarsità di dati etichettati: la maggior parte dei metodi di deep learning richiede grandi quantità di dati annotati manualmente, che sono costosi e difficili da reperire. Al contrario, i corpora di testo non etichettato sono abbondanti, rendendo fondamentale la capacità di apprendere informazioni linguistiche direttamente dal testo grezzo;
- sfruttare informazioni che vadano oltre il significato della singola parola;
- la necessità di un modello "task-agnostic" anziché "task-specific", ovvero di un modello in grado di apprendere una rappresentazione universale del linguaggio che possa essere trasferita a diversi task con pochissimi adattamenti.

L'approccio proposto da OpenAI si basa su una procedura di addestramento a due fasi: il pre-training non supervisionato e il fine-tuning supervisionato.

L'obiettivo del pre-training è apprendere i parametri iniziali di una rete neurale utilizzando un vasto corpus di testo non etichettato. Dal punto di vista matematico, ciò si traduce nel massimizzare la probabilità di prevedere il token successivo in una sequenza di testo basandosi su una finestra di contesto precedente. La funzione obiettivo (likelihood) da massimizzare è espressa dalla seguente formula:

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta),$$

dove $\mathcal{U}$ è l'insieme di dati non etichettati, k è la dimensione della finestra di contesto e Θ sono i parametri del modello.

Il modello utilizzato è il decoder dell'architettura Transformer, in modo da catturare dipendenze a lungo raggio tra i token.

Una volta completato il pre-addestramento, i parametri vengono adattati ad un task specifico utilizzando dati etichettati. Questo equivale a massimizzare la seguente likelihood:

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m),$$

dove $\mathcal{C}$ è il dataset etichettato (relativo ad un task specifico) e $x^1, \dots, x^m$ sono i token con etichetta y.

L'applicazione del paradigma GPT ha portato a progressi significativi in diverse aree del Natural Language Understanding, sia in termini di risultati quantitavi sui benchmark[1] che di "comprensione" sintattica e semantica trasferibile, dimostrando che un unico modello può eccellere in task diversi con modifiche minime all'architettura.

Come accennato all'inizio del paragrafo, GPT-1 non può essere considerato un vero e proprio LLM secondo gli standard moderni. Al giorno d'oggi, tale definizione è riservata a modelli che portano il numero di parametri ad una scala estrema, permettendo l'emergere di prestazioni nettamente superiori. Un esempio evidente è GPT-3, che presenta circa 175 *miliardi* di parametri (contro i 117 *milioni* nel caso di GPT-1).

3.2 Chatbot

I chatbot sono sistemi in grado di sostenere conversazioni prolungate con l'obiettivo di imitare le conversazioni non strutturate tipiche dell'interazione informale tra esseri umani [2].

E' dunque importante non confondere i termini "Large Language Model" e "Chatbot": mentre un LLM è l'architettura neurale addestrata a prevedere statisticamente la sequenza di parole più probabile, un chatbot è un'applicazione che rende quella potenza utilizzabile in una conversazione. In altre parole, un chatbot (o "LLM-based conversational system") modula l'output di un LLM applicando filtri e logica aggiuntiva, come istruzioni di sistema o gestione della memoria per dare continuità al dialogo.

Per chiarire tale distinzione, di seguito si riporta un semplice esempio che mette a confronto l'output di un LLM[2] e di un chatbot[3] in risposta allo stesso input:

[1] GPT-1 ha ottenuto un punteggio complessivo di 72,8 su GLUE (General Language Understanding Evaluation), superando il precedente record di 68,9.
[2] Gemini 3 Flash Preview [8] con *Temperature = 0*.
[3] Gemini App [9].

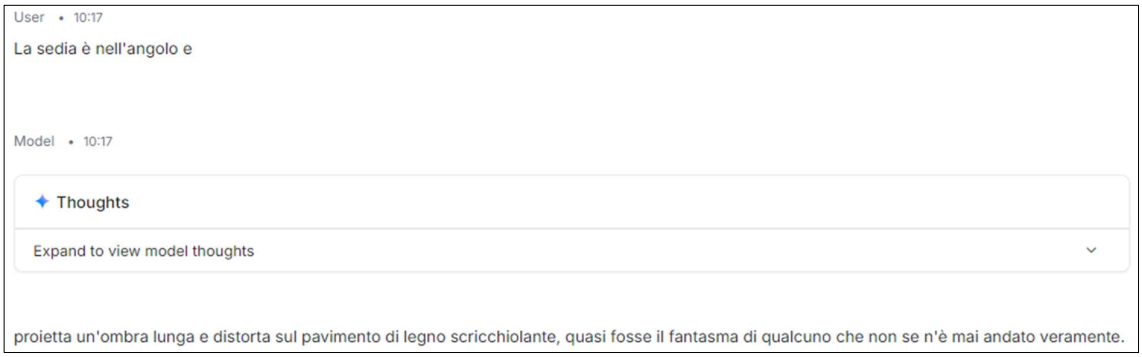

Fig. 7: esempio di output di un LLM.

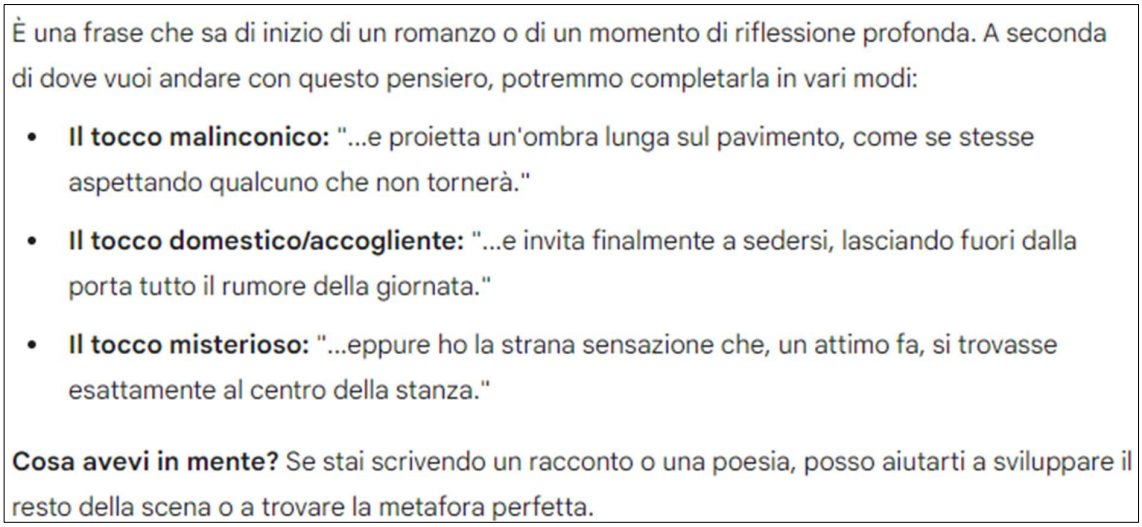

Fig. 8: esempio di output di un LLM-based conversational system.

Come si può osservare, mentre il LLM si limita a continuare la frase (ovvero a stimare le parole successive), il chatbot sembra "ragionare" sul significato del prompt ("*È una frase che sa di inizio di un romanzo o di un momento di riflessione profonda.*") e mostra la sua natura di assistente ("*[…] potremmo completarla in vari modi:*", "*Cosa avevi in mente? […] posso aiutarti a*").

3.3 Instruction Tuning

Un tratto distintivo dei chatbot moderni è la loro capacità di rispondere ai comandi (prompt) dell'utente. Per far sì che un modello linguistico sia allineato alle richieste umane, si ricorre ad una tecnica chiamata Instruction Tuning [5]. Questa tecnica consiste nel fine-tuning su dataset strutturati sotto forma di istruzioni. L'idea di base è che il modello impari a generalizzare l'esecuzione di un'istruzione mappandola sulle conoscenze apprese durante il pre-training.

Per implementare l'Instruction Tuning si trasformano i dataset NLP esistenti (come quelli per la traduzione, la sentiment analysis, ecc.) in un formato di istruzioni: per ogni task vengono creati manualmente diversi template che descrivono il task da svolgere in linguaggio naturale. In questo modo il modello non impara solo a risolvere il singolo task, ma anche ad interpretare la struttura semantica delle istruzioni. Di seguito si riportano alcuni esempi di template per il task di Natural Language Inference[4]:

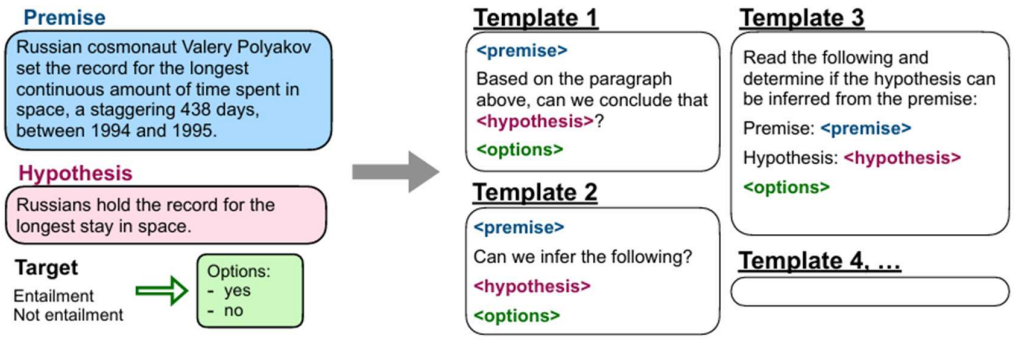

Fig. 9: template di istruzioni per descrivere un task NLI.

Come accennato, l'Instruction Tuning avviene contemporaneamente su molti task differenti.

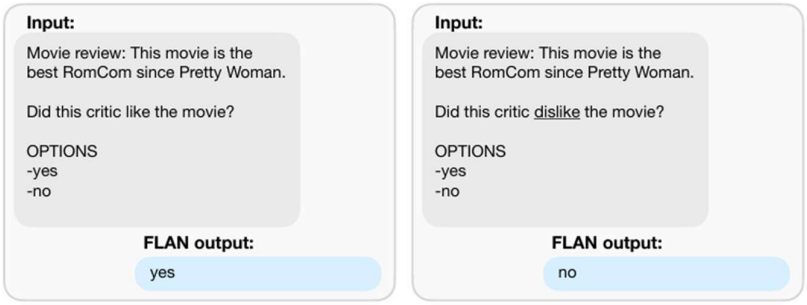

Fig. 10: output di FLAN[5] a diversi prompt di sentiment analysis.

[4] Il modello deve valutare se una premessa (Premise) implica logicamente un'ipotesi (Hypothesis). In questo esempio specifico il modello deve scegliere tra le due opzioni "yes" e "no", per cui è una semplificazione del task (che solitamente prevede le etichette "entailment", "neutral" e "contradiction").
[5] Finetuned Language Net [5]. E' un modello pre-addestrato di 137 miliardi di parametri che è stato utilizzato per il finetuning su più di 60 dataset di NLP espressi come istruzioni in linguaggio naturale.

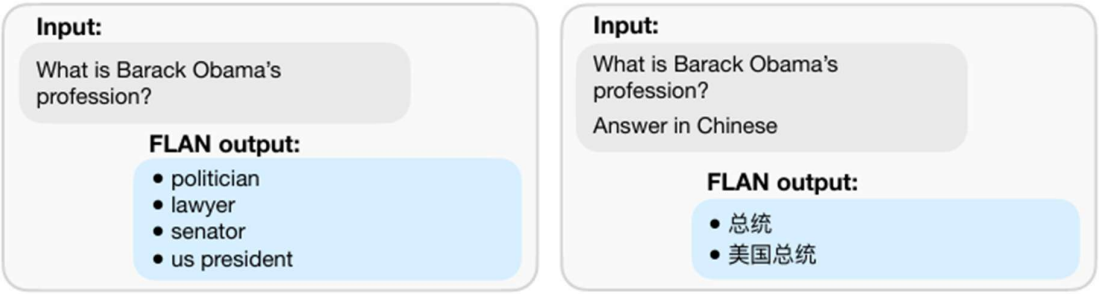

Fig. 11: output di FLAN a diversi prompt di question answering.

INPUT There are four ways an individual can acquire Canadian citizenship: by birth on Canadian soil; by descent (being born to a Canadian parent); by grant (naturalization); and by adoption. Among them, only citizenship by birth is granted automatically with limited exceptions, while citizenship by descent or adoption is acquired automatically if the specified conditions have been met. Citizenship by grant, on the other hand, must be approved by the Minister of Immigration, Refugees and Citizenship. Can we conclude that can i get canadian citizenship if my grandfather was canadian? OPTIONS: - no - yes
TARGET no

Fig. 12: esempio di Instruction Tuning per Reading Comprehension.

INPUT I packed up my belongings. What is the cause? OPTIONS: - I was hunting for a new apartment. - I was moving out of my apartment.
TARGET I was moving out of my apartment.

Fig. 13: esempio di Instruction Tuning per Commonsense Reasoning.

INPUT What season is the Northern Hemisphere experiencing when it is tilted directly toward the Sun? OPTIONS: - fall - winter - spring - summer
TARGET summer

Fig. 14: esempio di Instruction Tuning per Closed-Book QA.

INPUT How does the sentence end? Elena wanted to move out of her parents fast but Victoria wanted to stay for a while, OPTIONS: - Elena went to school. - Victoria went to school.
TARGET Victoria went to school.

Fig. 15: esempio di Instruction Tuning per Coreference Resolution.

E' importante sottolineare che al termine dell'Instruction Tuning il modello avrà acquisito capacità di generalizzazione zero-shot, diventando in grado di eseguire comandi relativi anche a task mai visti in precedenza.

In sintesi, l'Instruction Tuning migliora la capacità del modello di seguire comandi, ma non gli impedisce di generare output non veritieri, tossici o non utili.

3.4 RLHF

Il Reinforcement Learning from Human Feedback (RLHF) è una forma di fine-tuning che serve per allineare l'output di un LLM con le preferenze umane [6]. Più precisamente nasce con l'idea di rendere i modelli linguistici:

- di aiuto (*helpful*), ovvero in grado di aiutare l'essere umano a risolvere un compito;
- veritieri (*honest*), cioè tali da non inventare informazioni e da non sviare l'utente;
- innocui (*harmless*), cioè che non causino danni fisici, psicologici o sociali alle persone o all'ambiente.

Il RLHF si articola in 3 fasi:

1. SFT (Supervised Fine-Tuning): un modello pre-addestrato viene "adattato" in maniera supervisionata su un dataset di istruzioni etichettate manualmente da operatori umani;
2. Reward Modeling: il modello genera più possibili risposte per uno stesso prompt. Annotatori umani confrontano tali risposte e le ordinano in base alle preferenze (ranking), dalla "migliore" alla "peggiore". A partire da queste preferenze viene addestrato un modello di ricompensa (Reward Model), che impara ad assegnare un punteggio alle risposte in base al loro grado di allineamento con le preferenze umane;
3. Ottimizzazione tramite Reinforcement Learning: il modello linguistico viene trattato come una policy, ovvero come un agente che genera token per massimizzare una ricompensa. Questa policy viene ottimizzata tramite l'algoritmo PPO (Proximal Policy Optimization): la policy genera una risposta a un prompt e il Reward Model assegna un valore scalare come ricompensa. L'obiettivo dell'algoritmo è aggiornare i parametri della policy per massimizzare tale ricompensa. Per evitare il fenomeno del "reward hacking" (ovvero che il modello "inganni" il Reward Model con risposte formalmente corrette ma prive di senso) o che si discosti troppo dal linguaggio naturale, viene applicata una penalità basata sulla divergenza di Kullback-Leibler tra la policy corrente e il modello SFT iniziale.

Nonostante questo meccanismo di allineamento, però, i LLM presentano dei limiti noti come "allucinazioni".

4. Allucinazioni e strategie di mitigazione

Per “allucinazione” di un modello linguistico si intende la generazione di testo non supportato dalla fonte di input o fattualmente scorretto[6], nonostante risulti fluente e plausibile.

Esistono due categorie principali di allucinazione:

- allucinazioni intrinseche: l’output contraddice le informazioni presenti nell’input;
- allucinazioni estrinseche: l’output introduce informazioni non presenti o non supportate dall’input, che possono essere vere o false.

Per esempio, dato l’input “*Mario ha 20 anni*”:

- l’output “*Mario ha 30 anni*” è un’allucinazione intrinseca;
- l’output “*Mario ha 20 anni e vive a Roma*” è un’allucinazione estrinseca.

4.1 Cause delle allucinazioni nei chatbot

Nel caso dei chatbot basati su LLM, le allucinazioni possono derivare da diversi fattori strutturali legati alla natura del modello e alle modalità di generazione. Tra i principali si possono individuare:

- la conoscenza appresa durante il pre-training è statica e non viene aggiornata automaticamente. Di conseguenza, il modello può generare risposte basate su informazioni obsolete;

Fig. 16: esempio di risposta[7] basata su fonti non aggiornate[8].

[6] Falso rispetto alla realtà.
[7] La domanda è stata posta in data 2/03/2026.
[8] La modalità “Grounding with Google Search” di Google AI Studio è stata disattivata per evidenziare i limiti delle risposte basate solamente sulla conoscenza appresa durante il pre-addestramento, in questo caso obsoleta.

- errori di ragionamento implicito, che si verificano solitamente in compiti complessi che richiedono inferenze logiche e calcoli matematici. Dato che il processo inferenziale non è esplicitamente controllato, l'errore può verificarsi in un passaggio intermedio non esplicitato, compromettendo la correttezza del risultato finale. Esempio:

  Prompt: "*In una stanza ci sono 3 fratelli. Ognuno di loro ha una sorella. Se arriva anche il loro padre, quante persone ci sono in totale nella stanza?*"

  Risposta corretta: "*5 (i 3 fratelli, la sorella – che è la stessa per tutti - e il padre).*"

  Risposta del chatbot: "*7 (3 fratelli + 3 sorelle + 1 padre).*"

  In questo esempio l'allucinazione deriva da un'inferenza logica scorretta;

- assenza di pianificazione/strumenti esterni: la risposta è generata in un'unica fase, senza suddividere il compito in sotto-obiettivi e senza delegare operazioni specifiche a strumenti esterni specializzati. Per esempio, se chiedessimo ad un chatbot "*Qual è il totale delle vendite del trimestre dato questo dataset Excel?*", esso potrebbe stimare un numero approssimativo basandosi su pattern numerici appresi durante l'addestramento anziché effettuare il calcolo tramite un modulo esterno;
- output non revisionato: il modello può generare una risposta plausibile anche in condizioni di incertezza, poiché è ottimizzato per produrre output coerente, ma non per espirmere esplicitamente incertezza quando non dispone di informazioni sufficienti. Inoltre è importante sottolineare che in assenza di meccanismi di revisione dell'output, potrebbero emergere criticità legate all'etica[9] e alla sicurezza. Ad esempio, un modello potrebbe generare contenuti pericolosi o inappropriati in risposta a prompt sensibili.

Oltre ai fattori elencati, le allucinazioni possono anche derivare da criticità intrinseche nelle fasi di addestramento e inferenza:

- durante il Supervised Fine-Tuning (e nello specifico nell'Instruction Tuning), il modello viene addestrato a imitare le risposte umane. Questo significa che il modello impara a rispondere sempre in modo completo. Se però un prompt riguarda conoscenze che non sono state apprese durante il pre-training, il modello "simula" la conoscenza pur di soddisfare il compito richiesto;
- nel caso del RLHF, il Reward Model è addestrato su preferenze umane. Se però gli annotatori non sono esperti del dominio e prediligono risposte sintatticamente corrette e ben strutturate piuttosto che dichiarare l'incertezza, la policy del modello viene ottimizzata per massimizzare la ricompensa generando risposte convincenti ma false;
- nell'ultimo strato del Transformer, la distribuzione di probabilità ottenuta tramite l'applicazione della Softmax somma a 1. Questo significa che il modello

[9] Già con GPT-3 si era constatato che il modello rifletteva bias e pregiudizi presenti nei dati di Internet su cui il modello era stato addestrato. Ad esempio, nel task del sentiment associato a diverse categorie etniche, il modello attribuiva costamente un sentiment basso alla categoria "Black" [5].

selezionerà il token con punteggio più alto anche in condizioni di incertezza (in pratica “punteggio alto” diventa sinonimo di “verità”).

4.2 RAG

Per Retrieval-Augmented Generation (RAG) si intende una tecnica che combina un modello generativo dotato di memoria parametrica (Generator) con un modulo di recupero di informazioni da una memoria non parametrica esterna (Retriever) per la generazione del linguaggio [10]. In altre parole, il modello non si affida esclusivamente alla conoscenza appresa durante il pre-training, ma integra nel processo generativo informazioni recuperate dinamicamente da fonti esterne.

I modelli RAG sono nati per affrontare alcune limitazioni degli LLM, tra cui la staticità della conoscenza parametrica, che può risultare incompleta o obsoleta.

Quando un utente fornisce un prompt di input, un sistema RAG agisce secondo le seguenti fasi:

1. Retrieval: il prompt viene mappato in una rappresentazione vettoriale (embedding) e confrontato con un indice esterno (tipicamente un vector database[10]). Il Retriever seleziona i “top-K” documenti più rilevanti in base alla similarità semantica tra la query[11] e i documenti indicizzati;
2. Augmentation: i documenti recuperati vengono concatenati o integrati nel prompt originale come contesto aggiuntivo;
3. Generation: il Generator produce il testo finale basandosi sia sulla propria conoscenza interna (parametri) che sulle informazioni esterne appena recuperate.

[10] Struttura dati contenente documenti indicizzati. Ogni documento è memorizzato sotto forma di embedding.
[11] Tecnicamente i termini “prompt” e “query” si riferiscono a due tipi di input diversi: il prompt è l’input fornito dall’utente e serve al Generator. La query invece è una versione riformulata o ottimizzata del prompt che serve al Retriever per un recupero efficiente delle informazioni.

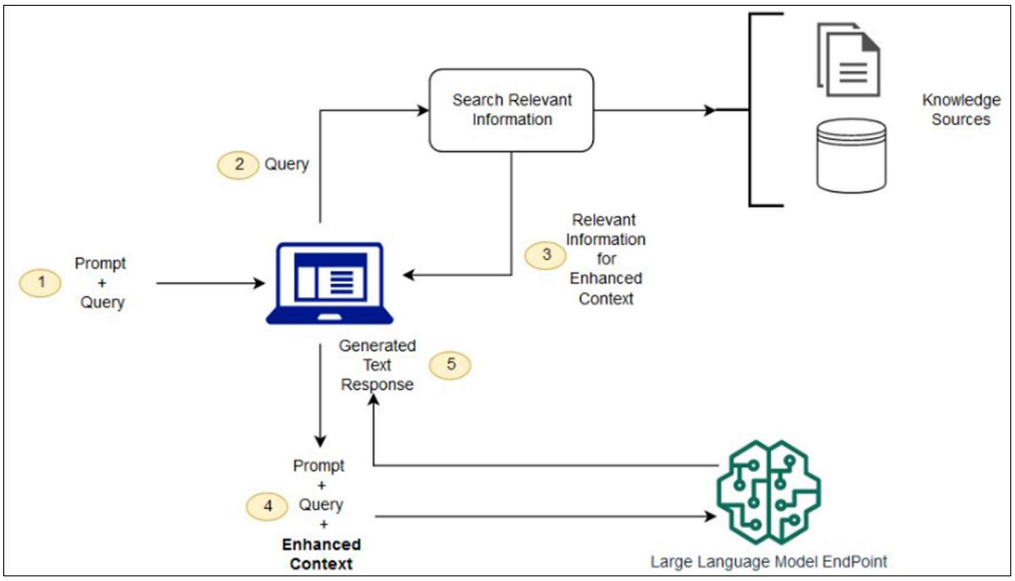

Fig. 17: funzionamento di RAG con gli LLM.

Se implementato tramite RAG, un modello dovrebbe fornire la seguente risposta al prompt riportato in *Fig. 16*:

Fig. 18: esempio di risposta basata su retrieval[12].

RAG offre dunque diversi vantaggi:

- riduce le allucinazioni dovute a fonti obsolete;
- rende possibile aggiornare la conoscenza del modello sostiuendo l'indice dei documenti anziché riaddestrando l'intero modello;

[12] La modalità "Grounding with Google Search" è stata attivata per consentire il recupero di informazioni aggiornate.

- fornisce una sorta di “provenienza” delle decisioni, poiché è possibile ispezionare i documenti che il modello ha consultato per generare una specifica risposta.

Tuttavia è bene tenere a mente che:

- nessuna fonte di conoscenza esterna è completamente priva di pregiudizi e inesattezze;
- è necessario aggiornare periodicamente l’indice esterno. Questo può essere fatto manualmente, in modo automatizzato tramite una pipeline ETL (Extract, Transform and Load) o in tempo reale con ricerca sul web. In ogni caso rimane un’operazione costosa da eseguire;
- non risolve le restanti criticità citate nel paragrafo 4.1.

4.3 Ragionamento multi-step

Il multi-step reasoning è la capacità di risolvere problemi complessi attraverso una sequenza di passaggi di ragionamento intermedi. Esso può essere implementato tramite varie tecniche, tra cui il Chain-of-Thought Prompting [11].

4.3.1 Chain-of-Thought Prompting

Il Chain-of-Thought (CoT) Prompting è una tecnica che serve ad un LLM per decomporre problemi complessi in passaggi più semplici e gestibili prima di fornire la risposta finale. Questo approccio si è rivelato molto efficiente per migliorare le prestazioni dei modelli per task aritmetici, di commonsense reasoning e di ragionamento simbolico.

Il CoT Prompting si realizza attraverso il few-shot prompting, ovvero fornendo tramite prompt ad un modello pre-addestrato alcuni esempi che mostrano il processo di ragionamento desiderato.

Diversamente dal prompting tradizionale, in cui vengono fornite coppie <input,output>, si passa al modello anche una spiegazione in linguaggio naturale che porta alla soluzione del problema. In questo modo un LLM impara a simulare lo stile di risposta fornito per i nuovi quesiti che gli verranno sottoposti.

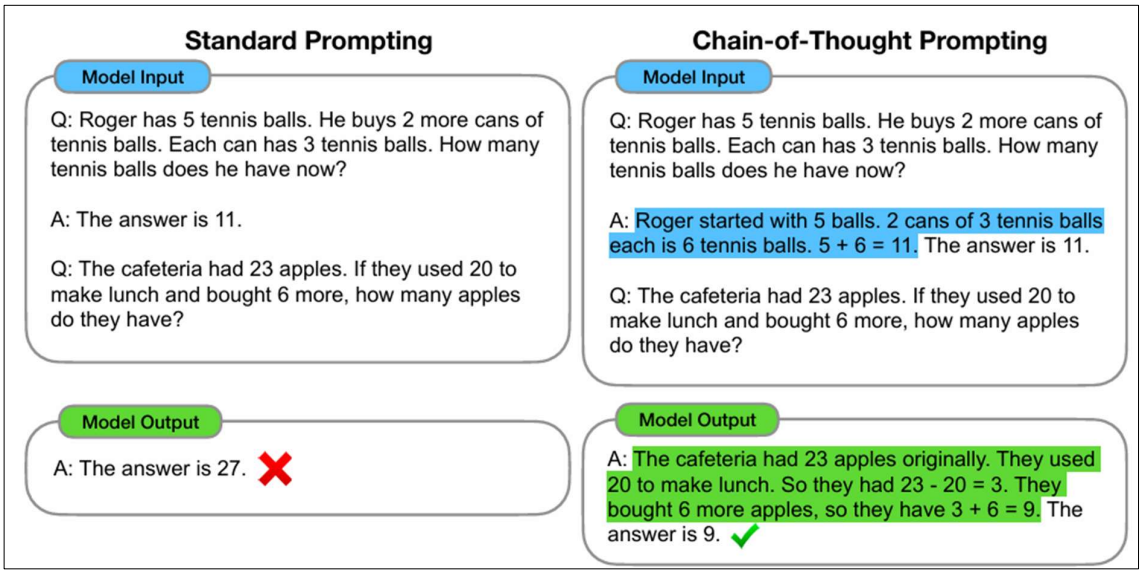

Fig. 19: esempio di Standard Prompting vs CoT Prompting. I processi di ragionamento CoT sono evidenziati.

Il CoT Prompting presenta diverse proprietà che facilitano il ragionamento nei modelli linguistici:

- dato che un problema complesso viene scomposto in più sotto-problemi, è possibile allocare risorse computazionali aggiuntive a problemi che richiedono un maggior numero di fasi di ragionamento;
- una CoT facilita l'interpretazione delle decisioni prese dal modello, il chè risulta particolarmente utile nel caso in cui la risposta fornita sia errata;
- non richiede il fine-tuning, ma sfrutta la capacità dei LLM di apprendere nel contesto ("In-Context Learning") semplicemente fornendo loro degli esempi di come "pensare" prima di rispondere.

Nonostante questi vantaggi, il CoT Prompting presenta anche degli aspetti negativi:

- è efficiente solo per modelli di grandi dimensioni (tipicamente con almeno 100 miliardi di parametri). Nei modelli più piccoli, invece, non produce miglioramenti e può persino portare alla generazione di catene di pensiero illogiche;
- i vantaggi del CoT sono minimi per task molto semplici che richiedono un solo passaggio logico ("SingleOp"). In questo caso risulta spesso più efficiente il prompting tradizionale;
- non garantisce che il percorso di ragionamento generato dal modello sia corretto: anche quando la risposta finale è esatta, il ragionamento potrebbe essere sbagliato a livello logico;

- allo stesso tempo non garantisce la correttezza della risposta: il modello potrebbe seguire una catena di pensiero corretta ma sbagliare un calcolo, fornendo il risultato errato.

4.3.2 Self-Consistency

La Self-Consistency è un'estensione del Chain-of-Thought. Mentre CoT prevede un singolo percorso di ragionamento, la Self-Consistency campiona diversi percorsi paralleli per risolvere lo stesso problema. L'idea alla base è che un problema di ragionamento complesso tipicamente può essere risolto tramite più processi logici che convergono verso un'unica risposta corretta [12].

Il funzionamento della Self-Consistency, noto come "Sample-and-Marginalize", consiste nei tre passaggi seguenti:

1. Prompting: si generano diverse istanze CoT sullo stesso prompt affinchè il modello espliciti diversi percorsi logici intermedi;
2. Sample (campionamento): invece di selezionare la risposta più probabile (approccio Greedy), si campionano diversi percorsi di ragionamento;
3. Marginalize: si seleziona come risposta definitiva quella più frequente tra i percorsi campionati tramite voto di maggioranza, "emarginando" i ragionamenti errati che tendono a non convergere verso lo stesso risultato.

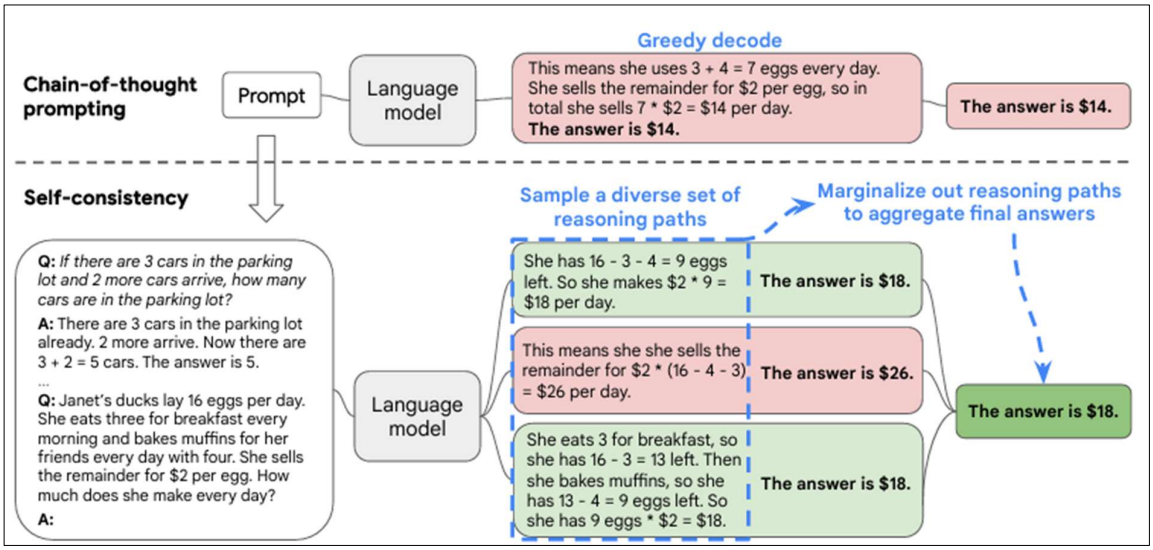

Fig. 20: esempio di CoT vs Self-Consistency rispetto allo stesso prompt.

In base alle tecniche di prompting utilizzate, al tipo di ragionamento intermedio e alle strategie di aggregazione dei risultati, la Self-Consistency può essere declinata in diverse forme.

In base al prompting, si possono distinguere le seguenti varianti:

- Few-shot Self-Consistency: è la forma “classica”, che utilizza alcuni esempi manuali nel prompt per istruire il modello;
- Zero-shot Self-Consistency: viene applicata in combinazione con lo zero-shot chain-of-thought.

Per quanto riguarda il tipo di ragionamento, invece, si distinguono:

- Natual-Language Reasonoing Paths: la forma standard che utilizza frasi in linguaggio naturale per descrivere i passaggi logici;
- Non-Natural-Language Paths: il metodo funziona anche se il modello genera solo passaggi matematici anziché spiegazioni verbali.

Dal punto di vista delle strategie di aggregazione (ovvero di come il modello sceglie la risposta finale dai diversi percorsi campionati) esistono le seguenti varianti:

- Unweighted Sum (voto di maggioranza): è la forma più semplice, che prevede di selezionare la risposta che appare più frequentemente tra tutti i percorsi generati;
- Normalized Weighted Sum: ogni risposta viene pesata in base alla sua probabilità logaritmica condizionata, normalizzata per la lunghezza della sequenza (numero di token);
- Unnormalized Weighted Sum: simile alla precedente, ma utilizza la probabilità “grezza” senza correzione per la lunghezza.

In termini architetturali, è utile specificare che la Self-Consistency non viene realizzata attraverso una modifica dell’architettura interna del modello linguistico, ma agisce come una strategia di decodifica stocastica a livello di inferenza. In altre parole, la Self-Consistency si colloca “al di sopra” di un modello linguistico pre-addestrato, risultando indipendente dall’architettura specifica. Mentre il CoT standard si affida spesso a una decodifica greedy (deterministica), la Self-Consistency sfrutta il campionamento (sampling) per generare una distribuzione di risposte diverse dallo stesso modello.

La Self-Consistency tipicamente aumenta l’accuratezza nei task di ragionamento complesso, rivelandosi migliore del CoT. Inoltre, il livello di coerenza tra le risposte (ovvero la percentuale di decodifiche che concordano con la risposta finale) è altamente correlato con l’accuratezza. Questo permette al modello di fornire una stima della propria confidenza.

Tuttavia, la Self-Consistency è stata originariamente progettata per problemi di ragionamento con risposte finali univoche. La sua applicazione a task che prevedono la generazione di testo libero (come la text summarization) risulta più complessa, in quanto richiede la definizione di metriche di coerenza semantica (come la Cosine Similarity tra embedding per raggruppare risposte simili nel significato anche se scritte

in modo diverso) o l'utilizzo di un modello esterno che identifichi la risposta più rappresentativa tra i percorsi campionati.

4.3.3 Tree of Thoughts

Tree of Thoughts (ToT) è un framework che permette di risolvere un problema tramite una ricerca in un albero, dove ogni nodo rappresenta una soluzione parziale o un passaggio logico intermedio [13].

La risoluzione di un problema tramite ToT avviene secondo i seguenti passaggi:

1. il problema viene suddiviso in passaggi intermedi chiamati "thoughts";
2. data una situazione corrente, il modello genera diverse proposte per il passo successivo;
3. il modello usa un'euristica per giudicare se un thought è "promettente", "inutile" o "impossibile" (self-evaluation);
4. l'albero viene esplorato tramite algoritmi di ricerca, come Breadth-First Search (BFS), che ad ogni passo mantiene un set di percorsi più promettenti, o Depth-First Search (DFS), che esplora prima il percorso più promettente ed effettua backtracking se lo stato corrente è giudicato irrisolvibile.

Rispetto ai metodi di reasoning visti in precedenza, ToT presenta diversi vantaggi:

- permette di esplorare più percorsi di ragionamento contemporaneamente;
- introduce la valutazione dello stato intermedio;
- intercetta precocemente rami di fallimento, evitando di proseguire in ragionamenti errati;
- ha dimostrato prestazioni più elevate in task che richiedono pianificazione strategica.

Di contro, richiede un costo computazionale (token generati) significativamente superiore, stimato tra le 5 e le 100 volte superiore rispetto a CoT.

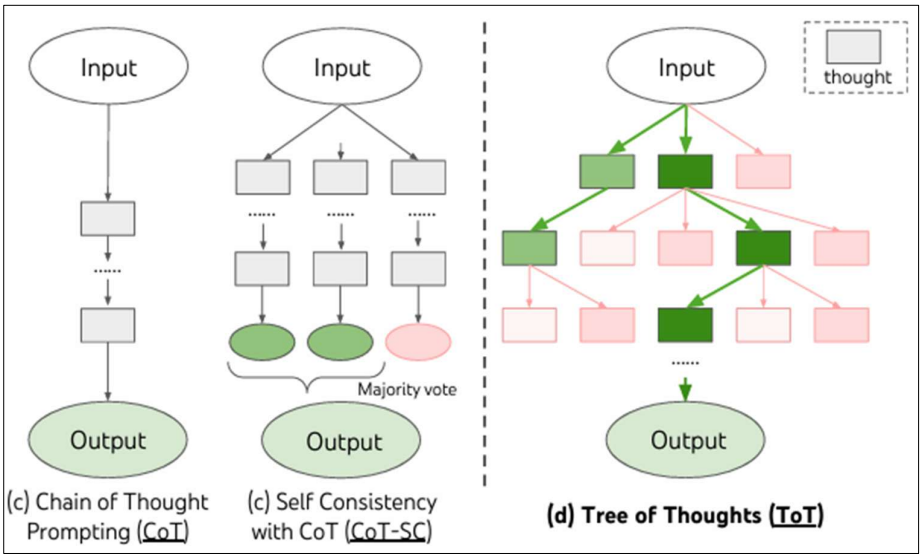

Fig. 21: confronto visivo tra Chain-of-Thoughts, Self-Consistency e Tree-of-Thoughts.

4.4 Sistemi agentici

In generale, un agente di IA Generativa è un'applicazione che cerca di raggiungere un obiettivo osservando il mondo e agendo su di esso attraverso l'uso di strumenti a sua disposizione [14].

Un agente è autonomo, cioè può agire indipendentemente dall'intervento umano una volta stabilito l'obiettivo da raggiungere. Inoltre è proattivo, ovvero è in grado di pianificare e selezionare le azioni più appropriate per raggiungere tale obiettivo anche in assenza di istruzioni esplicite.

Un agente opera attraverso un'architettura cognitiva, composta da tre elementi fondamentali:

- il modello: è il modello linguistico (LLM) responsabile dei processi di ragionamento e generazione del linguaggio. Può utilizzare strategie di reasoning (come Chain-of-Thoughts) per guidare il processo decisionale dell'agente;
- i tool: sono gli strumenti che permettono all'agente di interagire con dati e servizi esterni per compiere azioni che il solo modello non potrebbe eseguire, come interrogare database, effettuare ricerche sul web o accedere a informazioni aggiornate;
- il livello di orchestrazione: è il processo che coordina il ciclo operativo dell'agente, cioè stabilisce come vengono elaborate le informazioni, come il

modello effettua il ragionamento e quale azione debba essere presa fino al raggiungimento dell'obiettivo.

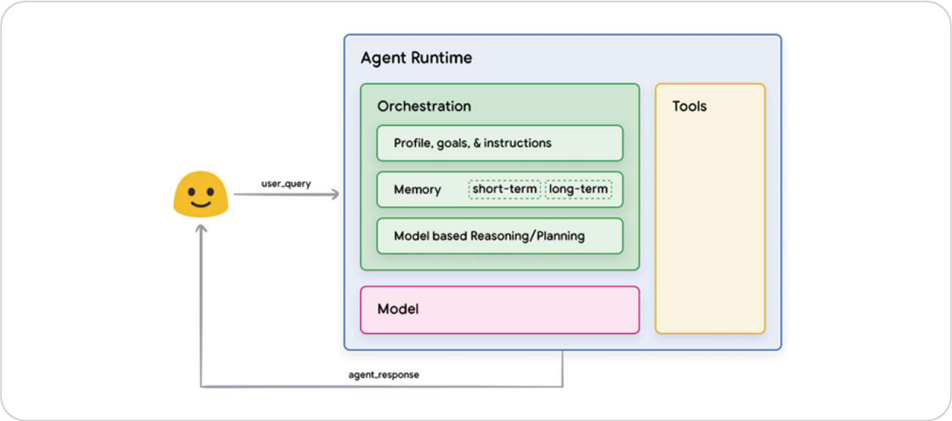

Fig. 22: architettura di un agente.

"Modello linguistico" e "agente" sono dunque concetti distinti: il modello è il "motore" cognitivo responsabile della generazione e del ragionamento, mentre l'agente è il sistema completo che utilizza quel motore per interagire con il mondo.

Mentre la conoscenza di un modello è solitamente limitata alle informazioni presenti nei dati di addestramento, un agente può estendere la propria conoscenza collegandosi a sistemi esterni tramite strumenti, permettendo risposte basate su dati attuali e specifici. Inoltre, un modello linguistico è tipicamente privo di memoria persistente tra interazioni diverse, mentre un agente gestisce nativamente la cronologia della sessione (ovvero la storia della chat nel caso dei chatbot), il chè gli permette di prendere decioni basate sui risultati di azioni precedenti.

Ricerche recenti hanno proposto diversi meccanismi basati sugli agenti per migliorare l'affidabilità dei LLM, come il ragionamento tool-augmented, la self-reflection e la generazione retrieval-grounded.

4.4.1 ReAct

ReAct è un paradigma che combina il ragionamento (Reasoning) e l'azione (Acting) all'interno dei LLM [15]. Il funzionamento di ReAct si basa su:

- Reason to Act: il modello genera dei "pensieri" ("reasoning traces") per indurre, monitorare e aggiornare i piani d'azione;
- Act to Reason: il modello esegue azioni che gli permettono di interfacciarsi con fonti esterne per raccogliere informazioni aggiuntive, che vengono poi incorporate nel processo di ragionamento per aggiornare la conoscenza del modello.

Il ciclo operativo di ReAct segue generalmente i seguenti passaggi:

1. Input: l'utente invia una richiesta (Question);
2. Thought: l'agente analizza la richiesta e pianifica il passaggio successivo;
3. Action: il modello seleziona uno strumento (Tool) e genera l'Action Input necessario;
4. Observation: l'agente riceve l'output dal mondo esterno (come il risultato di una query);
5. Update: il modello integra l'osservazione nel contesto, aggiorna il suo Thought e decide se ripetere il ciclo o fornire la risposta finale.

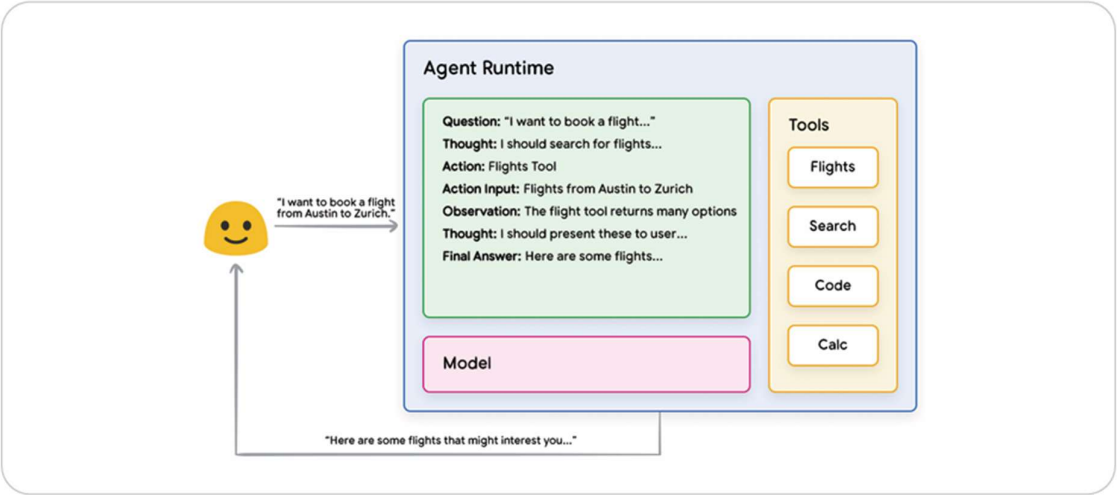

Fig. 23: esempio di funzionamento di ReAct.

ReAct si presta molto bene alla mitigazione delle allucinazioni, poiché è ancorato (*grounded*) al mondo esterno: i *Thoughts* generati vengono confrontanti e calibrati con la realtà delle *Observations*. In questo modo, la risposta finale non deriva solo dalla

conoscenza parametrica del modello (statica e talvolta obsoleta), ma anche da una conoscenza non parametrica aggiornata, che riduce la tendenza del modello a “inventare” fatti.

4.4.2 Self-Reflection

La Self-Reflection è un meccanismo che permette ad un agente di analizzare l’output prodotto e di identificare potenziali errori. In questo modo, l’agente può correggere il proprio reasoning e migliorare la risposta finale prima di presentarla all’utente.

Un framework basato sulla Self-Reflection è Reflexion [16]. Esso è stato progettato per rafforzare gli agenti basati su LLM attraverso il rinforzo verbale anziché mediante l’aggiornamento dei pesi del modello. In pratica Reflexion permette agli agenti di imparare dai propri errori attraverso un feedback in linguaggio naturale autogenerato.

Fig. 24: esempio di Verbal Reinforcement Learning.

Reflexion si basa su un processo iterativo di ottimizzazione che coinvolge tre componenti principali:

- Actor: un modello linguistico che genera traiettorie di azione e tentativi si soluzione basandosi sugli input dell’ambiente;
- Evaluator: esamina l’output dell’Actor e assegna un punteggio di performance (tipicamente un segnale binario di successo/fallimento);

- Self-Reflection: analizza la coppia <azione,punteggio> per generare un feedback verbale dettagliato. Questo feedback viene memorizzato in una memoria a lungo termine che l'Actor consulta nei tentativi successivi per evitare di ripetere gli stessi errori e ottimizzare il processo decisionale.

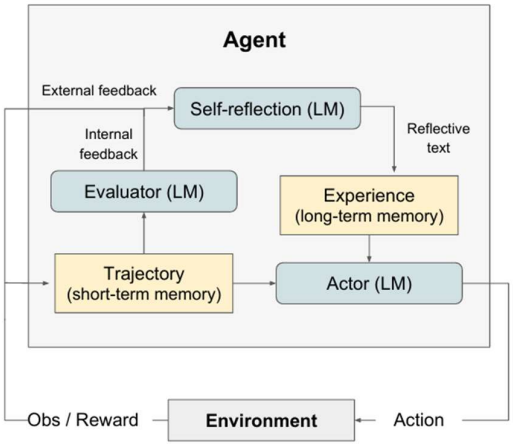

Fig. 25: architettura di Reflexion.

4.4.3 Self-RAG

Self-Reflective Retrieval-Augmented Generation (Self-RAG) è un framework che introduce un comportamento agentico all'interno del processo RAG (descritto nel paragrafo 4.2). A differenza del RAG tradizionale, in cui il retrieval viene eseguito in modo statico prima della generazione recuperando un numero fisso di documenti, il Self-RAG consente al modello di decidere dinamicamente se e quando effettuare il recupero delle informazioni [17].

Più precisamente, il Self-RAG prevede di addestrare un modello a decidere quando recuperare informazioni e a valutare criticamente i propri output utilizzando dei "Reflection Token", token speciali aggiunti al vocabolario del modello che permettono di controllare e valutare il processo di generazione. I Reflection Token si suddividono in quattro categorie principali:

- Retrieve (recupero): il modello decide se è necessario consultare fonti esterne per rispondere ("Yes", "No" o "Continue");
- ISREL (rilevanza): valuta se il passaggio recuperato fornisce effettivamente informazioni utili per risolvere il compito ("Relevant" o "Irrelevant");
- ISSUP (supporto): giudica il grado in cui l'affermazione generata è supportata dall'evidenza recuperata ("Fully supported", "Partially supported" o "No support");
- ISUSE (utilità): valuta l'utilità complessiva della risposta rispetto alla query originale su una scala da 1 a 5.

Al momento dell'inferenza, il funzionamento del Self-RAG si articola nelle seguenti 3 fasi:

1. recupero adattivo: per ogni segmento di testo da generare, il modello predice il token "Retrieve". Se il valore è "No", genera il testo normalmente. Se è "Yes", attiva un motore di ricerca per ottenere documenti rilevanti;
2. generazione in parallelo: se viene effettuato il recupero, il modello elabora più documenti in parallelo. Per ogni documento, il modello genera prima il token "ISREL" per determinare se il documento è un'effettiva fonte di conoscenza per la query; se è rilevante, genera un segmento di risposta basato esclusivamente sulle informazioni contenute in quel frammento;
3. critica e selezione: per ogni segmento generato, il modello produce i token "ISSUP" e "ISUSE". Infine, viene eseguita una segment-level beam search, che aggrega i punteggi di tutti i Reflection Token per selezionare, in ogni passaggio, la traiettoria di generazione che garantisce il massimo equilibrio tra fedeltà alla fonte e pertinenza alla domanda.

Per quanto riguarda il training, il framework Self-RAG prevede l'addestramento di due modelli distinti:

- Critic Model (C): viene addestrato inizialmente per inserire correttamente i Reflection Token in un corpus di addestramento;
- Generator Model (M): è il modello finale che impara a generare sia il testo del compito sia i Reflecton Token come se fossero parte del normale vocabolario, permettendogli di "auto-riflettere" in modo autonomo durante l'uso reale senza dover consultare il modello Critic.

In questo modo, Self-RAG integra retrieval, generazione e verifica all'interno di un unico modello, consentendo un controllo più esplicito sulla qualità e sull'affidabilità delle risposte generate.

4.4.4 Cooperazione tra agenti

Una direzione promettente per la riduzione delle allucinazioni è quella dei sistemi multi-agente. In questi sistemi, la scomposizione dei compiti tra agenti con ruoli distinti permette una sorta di "verifica reciproca" delle risposte generate, permettendo di individuare errori o incongruenze che un singolo agente potrebbe non rilevare.

Si possono identificare diversi paradigmi di interazione tra agenti:

- Role-Playing (gioco di ruolo): prevede di suddividere un problema complesso in sotto-task e di assegnare ognuno di questi ad un agente diverso (tipicamente la scomposizione e l'assegnazione sono svolte da un Manager Agent). In questo modo, ogni agente può applicare le proprie "competenze" specifiche al compito assegnato. Un sistema multi-agente che segue questo paradigma è

CAMEL [18], che utilizza l'Inception Prompting[13] per permettere ad un "Task Planner" e ad un "Task Executor" di interagire autonomamente. Questo loop riduce il carico cognitivo del singolo modello e minimizza le allucinazioni grazie alla specificità dei ruoli di ciascun agente;

- Critic-In-The-Loop: introduce un agente che svolge la funzione di "critico", ovvero ha il compito di revisionare le proposte degli altri agenti. Questo ablita un processo decisionale complesso simile ad una ricerca ad albero, dove ogni ramo viene valutato prima di procedere, scartando i percorsi che portano ad allucinazioni logiche;
- Multi-Agent Debate: prevede una sorta di "dibattito" tra agenti, in modo che ognuno di essi "difenda" una risposta diversa. L'idea è quella di costringere implicitamente il sistema a generare una risposta con confidenza più alta rispetto ad un singolo modello.

4.5 Limitazione dei contenuti

Nonostante le soluzioni agentiche introducano capacità avanzate di ragionamento, pianificazione e utilizzo di strumenti esterni, esse non eliminano completamente il rischio di allucinazioni o della generazione di contenuti inappropriati. Per questo motivo vengono spesso affiancate da ulteriori meccanismi di controllo, come review e guardrail.

La review consiste nella revisione critica della risposta generata dal modello prima che essa venga fornita all'utente. Questo processo può essere svolto dallo stesso modello (come nel caso di Reflexion descritto nel paragrafo 4.5.2) oppure da un secondo modello specializzato (in questo caso di parla di "Cross-Modell Review").

I guardrail invece impongono dei vincoli per limitare o filtrare i contenuti generati da un modello. Senza guardrail, i chatbot potrebbero generare contenuti indesiderati, pericolosi o eticamente inappropriati [19]. In pratica i guardrail agiscono come un livello di controllo tra il modello che genera una risposta e l'utente che ha effettuato una richiesta.

Rispetto alla generazione della risposta, i guardail possono essere applicati in tre momenti diversi:

- prima della generazione, per validare l'input dell'utente e rilevare linguaggio inappropriato, tentativi di prompt injection o altre forme di attacco;
- durante la generazione, per guidare il comportamento del modello e imporre vincoli sul tipo di contenuti che possono essere prodotti;
- dopo la generazione, per validare l'output e assicurarsi che la risposta finale non contenga contenuti tossici, distorti o allucinati.

[13] L'Inception Prompting è una tecnica di Prompt Engineering che ha l'obiettivo di fornire a due o più agenti un set di istruzioni iniziali ben strutturate in modo che essi possano collaborare tra loro senza ulteriore intervento umano, come se il compito fosse stato "innestato" nella loro logica di funzionamento.

I guardrail permettono inoltre l'applicazione di diverse policy di filtraggio, tra cui:

- filtri sui contenuti, per bloccare odio, insulti, violenza e contenuti sessuali;
- argomenti negati, per definire temi che il modello non deve affrontare;
- filtri sulle parole, per individuare e bloccare termini specifici;
- filtri su informazioni sensibili, per impedire l'esfiltrazione di dati riservati come le Personal Identifiable Informations.

L'implementazione di guardail per i LLM può avvenire principalmente attraverso due modalità [20]:

- embedded rails: guardrail integrati direttamente nel modello. Questa tecnica si basa sull'allineamento tramite tecniche come Instruction Tuning (paragrafo 3.3) e RLHF (paragrafo 3.4) con l'obiettivo di istruire il modello a seguire i valori umani e a rifiutare categorie specifiche di prompt durante la generazione;
- programmable rails: agiscono come un intermediario tra l'utente e il LLM. Questi binari sono indipendenti dal modello sottostante e permettono agli sviluppatori di definire regole interpretabili e personalizzabili senza dover riaddestrare il LLM principale.

Per implementare un guardrail specifico che validi l'input dell'utente si posso attuare diversi approcci:

- content filter: prevede l'identificazione (tipicamente tramite espressioni regolari per dati strutturati o liste di termini proibiti) di parole chiave specifiche all'interno del prompt dell'utente che suggeriscono contenuti potenzialmente indesiderabili;
- classificazione basata su LLM esterno: si utilizza un modello linguistico secondario addestrato specificamente come classificatore. Ad esempio, il modello Llama Guard [21] è stato addestrato per classificare come "safe" o "unsafe" i prompt dell'utente sulla base di specifiche categorie di rischio ("Violence & Hate", "Sexual Content", "Guns & Illegal Weapons", "Regulated or Controlled Substances", "Suicide & Self Harm" e "Criminal Planning");
- architettura a proxy con Execution Rails: un motore di runtime esterno (Proxy) si interpone tra l'utente e il LLM e agisce da filtro attivo: esso intercetta il prompt dell'utente e interroga degli Execution Rails (logiche di controllo indipendenti). Se questi ultimi rilevano una violazione, il processo di generazione viene interrotto prima ancora che il prompt raggiunga il LLM principale, sostituendo l'output con una risposta di sicurezza predefinita (come ad esempio *"Spiacente, questa richiesta viola le norme di sicurezza"*).

Conclusioni

In questo approfondimento i chatbot sono stati analizzati principalmente dal punto di vista della Text Generation. Tuttavia, i modelli più recenti vanno ben oltre questa singola capacità: molti di essi sono infatti multimodali, ovvero in grado di elaborare e integrare informazioni provenienti da diverse modalità, come testo e immagini, rappresentandole all'interno di uno stesso spazio latente condiviso. Questo evidenzia come il campo del NLP e dei modelli generativi sia in continua evoluzione.

La ricerca attuale si sta concentrando su diverse nuove frontiere, tra cui:

- Small Language Models (SLM) e Model Compression: l'implementazione di modelli con un numero ridotto di parametri ma altamente ottimizzati. L'obiettivo è consentire l'esecuzione direttamente sui dispositivi locali (Edge AI), garantendo maggiore privacy, latenza ridotta e una riduzione dei costi energetici;
- Test-Time Compute: lo spostamento delle risorse computazionali dalla fase di addestramento alla fase di inferenza. Questo permette al modello di "ragionare" maggiormente prima di generare una risposta, migliorando così le prestazioni in compiti logico-matematici complessi;
- IA Neuro-Simbolica: la branca dell'IA che ha l'obiettivo di unire la capacità di apprendimento dei Transformer con la precisione logica e la capacità di ragionamento dei sistemi simbolici. Questa frontiera aiuterebbe a mitigare le allucinazioni attraverso l'uso di vincoli logici espliciti.

Una delle sfide più rilevanti, però, non riguarda soltanto il miglioramento delle prestazioni di questi sistemi, ma anche la loro integrazione responsabile nella vita quotidiana, garantendo il rispetto della privacy degli utenti e dei principi etici.

Bibliografia e sitografia

[1] Weizenbaum, J. (1966). ELIZA—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, *9*(1), 36-45.

[2] D. Jurafsky, J. H. Martin, *Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, Third Edition (draft), Stanford University – University of Colorado at Boulder, 2024.

[3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

[4] Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training.

[5] Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., ... & Le, Q. V. (2021). Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.

[6] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, *35*, 27730-27744.

[7] Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., ... & Le, Q. V. (2021). Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.

[8] https://aistudio.google.com

[9] https://gemini.google.com

[10] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, *33*, 9459-9474.

[11] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, *35*, 24824-24837.

[12] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

[13] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, *36*, 11809-11822.

[14] Google. (2024). *Building Agents with LLMs: A Roadmap* [Whitepaper]. Google Cloud / Google Research. https://archive.org/details/google-ai-agents-whitepaper

[15] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2022, October). React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

[16] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., ... & Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, *36*, 46534-46594.

[17] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023, October). Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.

[18] Li, G., Hammoud, H., Itani, H., Khizbullin, D., & Ghanem, B. (2023). Camel: Communicative agents for" mind" exploration of large language model society. *Advances in neural information processing systems*, *36*, 51991-52008.

[19] https://aws.amazon.com/it/blogs/machine-learning/build-safe-and-responsible-generative-ai-applications-with-guardrails/

[20] Rebedea, T., Dinu, R., Sreedhar, M. N., Parisien, C., & Cohen, J. (2023, December). Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *Proceedings of the 2023 conference on empirical methods in natural language processing: system demonstrations* (pp. 431-445).

[21] Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., ... & Khabsa, M. (2023). Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*.